

Desarrollo de interfaz modular y procesamiento de datos en PHP

Objetivos del práctico:

1. Separar la estructura visual de la lógica utilizando funciones de inclusión de archivos.
2. Construir un formulario y capturar sus datos de forma segura utilizando superglobales.
3. Aplicar diseño responsivo (Mobile First) y marcado semántico estructural.

Instrucciones para el estudiante:

1. **Estructura de Archivos (Modularidad):**
2. Crea los siguientes cuatro archivos:
 - **header.php:** Debe contener la etiqueta `<header>`, la apertura del `<body>` y el `<head>` del documento HTML.
 - **footer.php:** Debe contener la etiqueta `<footer>` y el cierre del `<body>` y `<html>`.
 - **index.php:** Página principal que contendrá el formulario de contacto.
 - **procesar.php:** Página encargada de recibir y mostrar los datos.
3. **Requisitos de Frontend (index.php, header.php, footer.php):**
 - Emplea etiquetas semánticas obligatorias: `<header>`, `<main>`, `<section>`, `<footer>`, `<form>`, `<fieldset>`, `<label>`.
 - Crea un archivo CSS externo. Aplica la metodología **Mobile First**: los estilos base deben estar pensados para pantallas móviles (elementos apilados, ancho 100%). Utiliza al menos una regla `@media` (min-width: 768px) para adaptar el diseño a pantallas más grandes (por ejemplo, colocando los campos del formulario en columnas).
4. **Requisitos del Formulario (index.php):**
 - El formulario debe solicitar al menos tres datos (ej: Nombre, Correo, Mensaje).
 - Debe configurarse para enviar la información al archivo `procesar.php` mediante el método POST.
 - Utiliza `require_once` o `include` para integrar `header.php` en la parte superior y `footer.php` en la parte inferior.
5. **Procesamiento de Datos (procesar.php):**
 - Integra igualmente `header.php` y `footer.php`.
 - Utiliza la superglobal `$_SERVER['REQUEST_METHOD']` para verificar que la página haya sido accedida mediante POST.
 - **Si el método es POST:** Captura los datos utilizando la superglobal `$_POST` y muéstralos en pantalla estructurados dentro del `<main>`.
 - **Si el método NO es POST:** (Por ejemplo, si el usuario accede directamente a la URL en el navegador), la página no debe intentar procesar datos. Debe mostrar un mensaje de error claro indicando que el acceso denegado o que no se enviaron datos por el formulario.

Rúbrica de Evaluación

Dimensión	Avance destacado (10-9)	Avance significativo (8-7)	Avance moderado (6-5)	Avance escaso (4-3)	Avance mínimo (2-1)
Manejo de Peticiones y Seguridad	Verifica correctamente el método POST usando <code>\$_SERVER</code> . Muestra el mensaje de error exacto y gestiona de forma impecable el acceso directo vía GET.	Verifica el método POST y muestra error en GET, pero el manejo visual del error es básico o presenta detalles menores en la lógica.	Verifica el método POST, pero el manejo del error si es GET es incompleto o la página genera advertencias (warnings).	No verifica adecuadamente el método de envío, pero logra procesar los datos si llegan por POST asumiendo el flujo ideal.	No hay distinción de métodos. El script falla, expone errores graves o procesa directamente sin control de acceso.
Uso de Superglobales	Utiliza <code>\$_POST</code> de manera precisa basándose en los atributos <code>name</code> para extraer y mostrar todos los datos de forma estructurada y limpia.	Utiliza <code>\$_POST</code> para extraer todos los datos solicitados correctamente sin errores, aunque la lógica de impresión en el HTML pueda ser repetitiva.	Utiliza <code>\$_POST</code> para extraer datos, pero omite capturar algún campo o los atributos <code>name</code> tienen errores menores que afectan	Intenta usar <code>\$_REQUEST</code> en lugar de la superglobal específica, mezcla métodos, o presenta errores al imprimir las variables en pantalla.	No logra capturar ni mostrar la información del formulario enviado.

			una variable.		
Modularidad (Inclusión de archivos)	Separa el código en <code>header.php</code> y <code>footer.php</code> de forma óptima. Utiliza <code>require</code> o <code>include</code> sin romper la semántica y estructura del documento en ninguna vista.	Logra la separación y carga de archivos, pero existen detalles menores en la división de las etiquetas de apertura y cierre estructural entre módulos.	Separa el código, pero existen etiquetas HTML duplicadas, mal cerradas o usa la inclusión en un lugar incorrecto que altera levemente el DOM.	Separa solo una parte del código (ej. solo el header) o el contenido de los módulos incluidos no tiene cohesión lógica.	El código se encuentra mayormente en archivos monolíticos o la inclusión falla impidiendo la carga correcta de la vista.
Principios de frontend trabajados	Usa correctamente HTML semántico. El CSS sigue estrictamente el enfoque Mobile First con media queries (<code>min-width</code>) precisas para adaptarse a escritorio.	Usa HTML semántico adecuado. El CSS es responsivo y aplica Mobile First, pero las reglas de media queries o la disposición en columnas podrían optimizarse.	Usa HTML semántico, pero el CSS es responsivo sin seguir estrictamente el orden de Mobile First (ej. construye para escritorio y adapta con <code>max-width</code>).	Faltan etiquetas semánticas clave (abuso de <code><div></code>). El diseño tiene CSS básico pero la adaptabilidad a diferentes pantallas es defectuosa.	No utiliza etiquetas semánticas ni implementa reglas de CSS responsivo en lo absoluto.